

PROTEA

Plataforma abierta de anotación funcional de proteínas
basada en modelos de lenguaje y re-ranking explicable

Documento de proyecto — versión para revisión interna

Francisco Miguel Pérez Canales
Universidad de Sevilla
`frapercan1@alum.us.es`

Abril 2026

Resumen

PROTEA es una plataforma abierta para la predicción de función proteica mediante modelos de lenguaje de proteínas, búsqueda por vecinos más cercanos (KNN) sobre embeddings vectoriales, y un re-ranker explicable que incorpora información de alineamiento y taxonomía. El proyecto responde a dos objetivos que la comunidad de anotación funcional todavía no ha resuelto conjuntamente: (1) obtener predicciones competitivas con el estado del arte en el benchmark de referencia (CAFA), y (2) hacerlo desde una plataforma reproducible, trazable y usable por investigadores sin perfil técnico. PROTEA se sitúa actualmente en el segundo puesto del *leaderboard* público de CAFA 6 (a la espera del cierre oficial del *ranking* privado) y está preparado para ser desplegado como servicio estable. Este documento describe el enfoque metodológico, la arquitectura técnica, el estado actual del proyecto, los recursos necesarios para el despliegue y la hoja de ruta de publicación. Se entrega como base para discusión y revisión interna con colaboradores.

Índice

1. Motivación	2
2. Estado del arte	2
3. Enfoque metodológico	3
3.1. Representación: embeddings reproducibles	3
3.2. Búsqueda KNN sobre corpus anotado	3
3.3. Re-ranker con información estructural y taxonómica	3
3.4. Entrenamiento del re-ranker sobre <i>holdout</i> temporal	4
4. Arquitectura técnica	4
4.1. Pilar 1 — Embeddings versionados como ciudadanos de primera clase	4
4.2. Pilar 2 — Operaciones como funciones puras del dominio	5
4.3. Pilar 3 — Sistema de colas heterogéneo con coordinación de dominio	5
5. Estado actual del proyecto	6
5.1. Resultados	6
5.2. Infraestructura de software	6
5.3. Trabajo pendiente a corto plazo	6

6. Infraestructura de despliegue	7
6.1. Situación actual: despliegue doméstico	7
6.2. Requisitos	7
6.3. Despliegue federado y ausencia de requisito GPU	7
6.4. Encaje con LifeWatch ERIC	8
7. Plan de publicación	8
8. Cronograma orientativo	8
9. Colaboradores y autoría	9
10. Cierre	9

1 Motivación

La anotación funcional de proteínas —la asignación de términos de Gene Ontology (GO) a secuencias proteicas— es un problema central en biología computacional. A medida que la secuenciación masiva produce proteomas de organismos no-modelo a un ritmo que supera con mucho la capacidad de anotación experimental, la predicción automática de función se ha convertido en un cuello de botella real para la biología de biodiversidad, la metagenómica y la genómica comparada.

Las herramientas clásicas del campo (InterProScan, eggNOG-mapper, PANNZER2) dependen fundamentalmente de búsquedas basadas en similitud de secuencia, modelos ocultos de Markov (HMMs) y ortología. Estos métodos funcionan bien en organismos bien representados en las bases de datos de referencia, pero pierden potencia cuando se aplican a especies no-modelo, proteomas recién ensamblados, o dominios de secuencia divergentes.

Desde 2020, los modelos de lenguaje de proteínas (ESM, ProstT5, Ankh, etc.) han demostrado que los embeddings vectoriales derivados de estos modelos capturan información estructural y funcional que no es accesible a los métodos clásicos. Sin embargo, la traducción de esa capacidad en herramientas prácticas para la comunidad sigue siendo desigual: la mayoría de sistemas publicados son *scripts* de investigación que requieren conocimiento técnico, no ofrecen interfaces usables y no garantizan reproducibilidad bit a bit entre ejecuciones.

PROTEA se diseña para cerrar esa brecha: predicciones competitivas con el estado del arte, expuestas a través de una plataforma web reproducible, auditable y usable sin conocimientos de línea de comandos.

2 Estado del arte

El estándar de evaluación del campo es la competición internacional CAFA (*Critical Assessment of protein Function Annotation*), que se celebra aproximadamente cada tres años y que establece el *benchmark* público más riguroso para predicción de función. Se organiza sobre un *holdout* temporal de anotaciones experimentales: los sistemas predicen función para un conjunto de proteínas cuya anotación real todavía no se conoce públicamente en el momento del envío, y se evalúan meses después cuando las anotaciones se hacen públicas.

Los participantes de referencia en CAFA 5 y CAFA 6 incluyen:

- **Herramientas clásicas** basadas en búsqueda de similitud y anotación por transferencia: InterProScan (consorcio InterPro, basado en perfiles HMM de múltiples bases), eggNOG-mapper (anotación por ortología a partir de la base eggNOG) y PANNZER2 (predicción bayesiana sobre alineamientos BLAST+ con ponderación de evidencias).
- **Sistemas basados en aprendizaje profundo sobre secuencias:** DeepGOPlus, NetGO 2.0/3.0, GOPredSim, entre otros. Estos sistemas combinan embeddings, redes convolucionales o modelos basados en atención con clasificadores supervisados sobre el vocabulario GO.
- **Sistemas mixtos** que combinan evidencias heurísticas de alineamiento, ortología y embeddings, típicamente ensamblados mediante *stacking* manual.

El trabajo presentado en este documento se posiciona en una tercera vía: usar embeddings de proteínas como representación primaria, pero sustituir el clasificador supervisado por una búsqueda KNN sobre un corpus de referencia anotado, y a continuación aplicar un re-ranker explicable que re-ordena los candidatos incorporando información de alineamiento y taxonomía. Esta combinación preserva la capacidad discriminativa de los modelos de lenguaje y añade una capa de interpretabilidad de la que los clasificadores monolíticos carecen.

3 Enfoque metodológico

3.1 Representación: embeddings reproducibles

Cada secuencia del corpus de referencia se codifica en un vector mediante un modelo de lenguaje de proteínas. PROTEA soporta actualmente cuatro backends (ESM-C, ProST5, Ankh y T5 clásico) con normalización unificada de tokens especiales, de forma que los índices posicionales en los embeddings coinciden exactamente con las posiciones de aminoácido.

La novedad práctica a nivel de sistema es que cada *receta* de embedding —modelo, versión, *chunking*, precisión numérica— se materializa como un objeto `EmbeddingConfig` identificado por UUID, que actúa como clave primaria de los vectores almacenados en la base de datos. Esto garantiza reproducibilidad bit a bit: regenerar dos veces los embeddings de la misma secuencia bajo el mismo `EmbeddingConfig` produce vectores idénticos.

3.2 Búsqueda KNN sobre corpus anotado

Dada una secuencia de consulta embebida, PROTEA recupera los k vecinos más cercanos en el corpus de referencia mediante búsqueda por similitud vectorial. El corpus de referencia se construye a partir de instantáneas de Gene Ontology Annotation (GOA) y QuickGO, con filtrado por fecha para permitir evaluaciones con *holdout* temporal estricto (críticas para el realismo del *benchmark*).

Cada vecino contribuye su conjunto de términos GO asignados al ranking de predicciones de la consulta, ponderado inicialmente por distancia en el espacio de embeddings.

3.3 Re-ranker con información estructural y taxonómica

La mejora clave respecto a un KNN puro consiste en aplicar un re-ranker posterior que incorpora dos fuentes de información adicionales:

1. **Alineamiento de secuencia:** para cada par (consulta, vecino) se calcula un alineamiento global (Needleman-Wunsch) y local (Smith-Waterman) mediante la librería `parasail` con matriz BLOSUM62, extrayendo identidad, similitud, cobertura, huecos y longitud del alineamiento.
2. **Información taxonómica:** se extrae la distancia taxonómica entre la especie de la consulta y la de la referencia (LCA, distancia, ancestros comunes, relación taxonómica clasificada) usando la taxonomía NCBI.

El re-ranker es un modelo supervisado (*gradient boosting*) que combina todas las *features* anteriores—distancia en embeddings, métricas de alineamiento, información taxonómica— para producir un *score* final por término GO candidato. Este planteamiento tiene dos ventajas claras sobre los sistemas basados exclusivamente en embeddings:

- **Explicabilidad intrínseca:** cada predicción puede justificarse mostrando al usuario la proteína de referencia que la motivó, el alineamiento que comparte con la consulta, y la relación taxonómica entre ambas. Esto es especialmente relevante en anotación funcional, donde el biólogo final necesita poder evaluar la credibilidad de la predicción antes de aceptarla.
- **Robustez ante *drift* de dominio:** cuando la consulta pertenece a un organismo mal representado, el re-ranker puede compensar mediante las *features* de alineamiento, recuperando capacidad predictiva donde un clasificador monolítico falla silenciosamente.

3.4 Entrenamiento del re-ranker sobre *holdout* temporal

El re-ranker se entrena sobre parejas generadas a partir de instantáneas históricas de GOA: se usa una ventana de lanzamientos anteriores como *train* y un lanzamiento posterior como *holdout* (actualmente GOA 160–225 para entrenamiento, GOA 230 como holdout). Este protocolo imita exactamente el escenario de evaluación de CAFA —predecir anotaciones que todavía no se conocen en el momento de entrenamiento— y garantiza que los resultados medidos en CAFA no están inflados por fugas temporales.

4 Arquitectura técnica

PROTEA se implementa como una plataforma completa que incluye base de datos, sistema de colas, API REST e interfaz web. La arquitectura está organizada en torno a tres pilares de diseño que merecen mención explícita porque son los que diferencian PROTEA de las herramientas clásicas del campo y los que van a estructurar la eventual publicación de ingeniería.

4.1 Pilar 1 — Embeddings versionados como ciudadanos de primera clase

En la mayoría de herramientas bioinformáticas que usan embeddings, los vectores se almacenan en archivos sueltos, con nombres basados en convenciones informales, y la *receta* usada para generarlos queda implícita en el código que los produjo. Cuando se reproduce un experimento meses después, a menudo es imposible saber con certeza con qué modelo, con qué parámetros y con qué versión de la librería se generaron los vectores originales.

PROTEA promueve la configuración de embedding a entidad de primera clase en la base de datos (`EmbeddingConfig`), identificada por UUID y referenciada como clave foránea por cada vector almacenado. Esto permite, entre otras cosas:

- Regenerar determinísticamente cualquier vector a partir de su `EmbeddingConfig` y la secuencia original.

- Comparar predicciones entre configuraciones distintas sin ambigüedad sobre qué se está comparando exactamente.
- Migrar entre precisiones numéricas (por ejemplo, de `fp32` a `halfvec/fp16`) con trazabilidad completa del cambio.

4.2 Pilar 2 — Operaciones como funciones puras del dominio

La lógica de dominio de PROTEA se organiza en torno al protocolo `Operation`, que exige que cada unidad de trabajo implemente una firma simple: un nombre y una función `execute(session, payload, *, emit)` que recibe una sesión de base de datos, la carga útil del trabajo y un *callback* para reportar progreso. Las operaciones no gestionan conexiones a RabbitMQ, no abren sesiones de base de datos por su cuenta, y no conocen la infraestructura de colas subyacente.

Esta separación es una aplicación directa de los principios de *hexagonal architecture* [1] y *clean architecture* [2] al dominio del *research software* bioinformático, donde la norma histórica han sido *workers* monolíticos que acoplan SQL, AMQP, lógica de dominio e inferencia en clases de varios cientos de líneas. No se reclama aquí la invención del patrón —existe en ingeniería de software desde al menos 2005— sino su aplicación sistemática a un campo que tradicionalmente no lo usa, y la demostración empírica de que esa disciplina habilita testabilidad sin infraestructura, extensibilidad, reutilización y reproducibilidad que los sistemas monolíticos no ofrecen.

4.3 Pilar 3 — Sistema de colas heterogéneo con coordinación de dominio

El tercer pilar es donde se concentra la contribución técnica original de PROTEA. El sistema de colas enruta trabajos a siete colas distintas según su naturaleza, y combina dos estilos de *worker* en el mismo *framework*:

- `QueueConsumer`, que gestiona transiciones de estado y filas de `Job` completas en la base de datos, apropiado para trabajos de larga duración que requieren auditoría.
- `OperationConsumer`, que procesa cargas útiles directas sin crear filas de `Job`, apropiado para micro-trabajos de alto volumen (por ejemplo, cada *batch* de inferencia dentro de un *job* mayor).

Sobre estos dos consumidores se implementan tres patrones que, por lo que conocemos, no están presentes en la literatura de *research software* bioinformático:

1. **Coordinadores con *backpressure* de dominio.** Las operaciones que dependen de recursos escasos (típicamente la GPU) levantan una excepción `RetryLaterError` cuando el recurso no está disponible; el coordinador atrapa la excepción, re-encola el trabajo con retardo y libera el *worker*. Esto expresa la contrapresión a nivel de dominio, no de infraestructura, y permite serialización natural del acceso a GPU sin necesidad de semáforos externos ni *locks* distribuidos.
2. **Jerarquía *parent-child* con progreso atómico.** Un trabajo padre (por ejemplo, *calcular embeddings para un proteoma de 50.000 secuencias*) se descompone en centenares de trabajos hijos que no crean fila de `Job` propia, sino que incrementan atómicamente el contador de progreso del padre. Esto reduce el volumen de escritura en la tabla de `Job` en uno o dos órdenes de magnitud sin perder visibilidad del progreso.
3. **Despliegue federado de *workers* bajo demanda.** Los *workers* no necesitan vivir en la misma máquina que el plano de control (base de datos, cola, API): basta con que tengan acceso de red al RabbitMQ central. Esto habilita topologías híbridas donde el plano de control reside en un servicio persistente (VM, *cloud*, servidor institucional) y los *workers* pesados

se lanzan como *jobs* en infraestructuras externas (HPC con SLURM, nodos *cloud* efímeros, estaciones de trabajo con GPU) sin tocar una línea del plano de control.

Este tercer pilar es, en términos de contribución original, el más fuerte del proyecto, y el que justifica la aspiración a publicar la parte de ingeniería en una revista de alto impacto.

5 Estado actual del proyecto

5.1 Resultados

- **Posición en CAFA 6:** actualmente #2 en el *leaderboard* público, a la espera del cierre oficial del *ranking* privado.
- **Comparación con el estado del arte:** se han obtenido métricas bajo el mismo marco de evaluación (el oficial de CAFA) para InterProScan, eggNOG-mapper y PANNZER2 sobre el mismo conjunto de validación, lo que permite una comparación directa y justa. Los resultados preliminares sostienen que el enfoque basado en embeddings + re-ranker supera claramente a los métodos basados en HMM y ortología en los tres aspectos de GO (BPO, MFO, CCO). A diferencia de las aproximaciones habituales, el mecanismo de evaluación y validación está *integrado de forma transparente en la propia plataforma*: el marco CAFA se ejecuta como parte del *pipeline* sobre las mismas estructuras de datos y el mismo flujo de trabajo que la inferencia, sin *scripts* externos ni post-proceso manual, garantizando que cada conjunto de predicciones es reproducible y comparable extremo a extremo.
- **Plataforma desplegable:** código base estable, con una *suite* de tests amplia y cobertura notablemente por encima del umbral habitual en *research software*, documentación Sphinx al día y *pipeline* completo funcionando desde la carga de un FASTA por el usuario hasta la descarga del TSV de predicciones.

5.2 Infraestructura de software

- **Stack:** FastAPI (API REST), SQLAlchemy 2.0 (ORM), PostgreSQL 16 con extensión pgvector, RabbitMQ (sistema de colas), Next.js 16 con Tailwind v4 (interfaz web).
- **Orquestación:** `docker compose` para desarrollo y despliegue local; imágenes publicadas automáticamente en *ghcr.io* en cada *push* a *main*.
- **Gestión de procesos:** `script manage.sh` que arranca, para, escala y monitoriza los 9 procesos (API, *workers* especializados, *frontend*) en desarrollo local.

5.3 Trabajo pendiente a corto plazo

- Iteración final limpia de la experimentación con traza completa de hiperparámetros, *datasets* y reproducibilidad verificable.
- Reconstrucción del método que obtuvo la posición en CAFA 6 en un *pipeline* documentado y reproducible de principio a fin (requisito imprescindible para la publicación del método).
- Ajustes finales de documentación y *tutorials*.
- Publicación de *preprint* en bioRxiv como paso previo al envío a revista.

6 Infraestructura de despliegue

6.1 Situación actual: despliegue doméstico

En el momento de redactar este documento, PROTEA se ejecuta íntegramente sobre el equipo personal del autor, expuesto públicamente mediante un túnel *ngrok* sobre un subdominio reservado. Toda la plataforma —API, PostgreSQL con *pgvector*, RabbitMQ, *workers* de cómputo, interfaz web y documentación Sphinx— corre en esta única máquina. El *stack* se arranca con `bash scripts/manage.sh start` y el túnel con un *script* auxiliar, quedando accesible desde Internet sin intervención de un administrador de sistemas.

Las características del equipo son:

Componente	Descripción
CPU	Intel Core i5-12400F (6 núcleos / 12 <i>threads</i>)
RAM	64 GB DDR4
GPU	NVIDIA GeForce RTX 3060, 12 GB VRAM
Disco	NVMe 1 TB
Sistema	Ubuntu 24.04 LTS (kernel Linux 6.17)
Red	Fibra doméstica + túnel <i>ngrok</i> con subdominio reservado

El uso observado durante las cargas de trabajo habituales —generación de *embeddings*, inferencia, *benchmarking* del re-ranker— es intensivo pero sostenible: la GPU se mantiene por debajo del 90 % de utilización y dentro del rango térmico seguro (del orden de 65 °C durante la inferencia, lejos del umbral de *throttling*), la CPU estabiliza el *package* en torno a 60 °C bajo carga continua, el almacenamiento NVMe se mantiene a ~33 °C y la ocupación del disco ronda el 70 %. En otras palabras, la máquina es capaz de ejecutar la plataforma extremo a extremo, pero opera cerca del techo razonable para un equipo doméstico no diseñado para servicio 24/7: cualquier uso externo sostenido requiere trasladar el servicio a una máquina de alojamiento estable.

6.2 Requisitos

Para desplegar PROTEA como servicio accesible se necesita una máquina —física o virtual— con las siguientes características mínimas:

Recurso	Mínimo	Recomendado
vCPU	4	8
RAM	16 GB	32 GB
Disco SSD	500 GB	1 TB
Sistema	Linux (Ubuntu 22.04 / Debian 12), Docker, <code>docker compose</code>	
Red	IP pública o subdominio HTTPS, puertos 80/443 accesibles	
Acceso	SSH administrativo, salida a Internet (descargas UniProt / GO)	
GPU	No requerida (ver más abajo)	

6.3 Despliegue federado y ausencia de requisito GPU

La arquitectura de PROTEA separa deliberadamente el *plano de control* —base de datos, sistema de colas, API, interfaz web, todos persistentes y de consumo moderado— del *plano de datos* —*workers* de inferencia con GPU, efímeros y desplegables bajo demanda en infraestructuras externas. La máquina de alojamiento no necesita GPU.

Los *workers* de cómputo pesado están diseñados para ejecutarse como procesos independientes en máquinas separadas que conectan con el RabbitMQ central, reciben los trabajos, los procesan y escriben los resultados directamente en la base de datos. Esto incluye tres modalidades ya contempladas en el diseño actual:

1. **Equipos propios del investigador** (GPU local disponible durante el trabajo diario).
2. **Despliegue bajo SLURM en centros HPC externos** —por ejemplo, CESGA (Centro de Supercomputación de Galicia) vía sus convocatorias de acceso competitivo. Los *workers* se empaquetan como imagen compatible (Singularity/Apptainer) y se lanzan como *jobs* SLURM que, una vez activos, se conectan al RabbitMQ central. El número de *workers* se escala pidiendo más *jobs* SLURM, sin tocar el plano de control.
3. **Otras infraestructuras *cloud* o Kubernetes** (EOSC, BSC Life Sciences, nodos institucionales) bajo el mismo modelo.

Esto significa que la contribución de una infraestructura *host* —una máquina modesta con alojamiento persistente y acceso externo— es precisamente el recurso que los centros HPC no pueden ofrecer, y viceversa. Ambos roles son complementarios y la arquitectura los coordina sin reconfiguración.

6.4 Encaje con LifeWatch ERIC

LifeWatch ERIC sostiene la infraestructura de investigación europea para biodiversidad y ecosistemas. PROTEA cubre una necesidad concreta poco atendida en ese ámbito: la anotación funcional de proteomas de organismos no-modelo, donde las herramientas clásicas pierden potencia por la escasa representación de estas especies en las bases de datos de referencia. Grupos de biodiversidad que trabajan con proteomas recién ensamblados —metagenómica, genomas nuevos, transcriptomas de especies no caracterizadas— obtienen con PROTEA una anotación robusta sin necesidad de conocimiento técnico ni infraestructura computacional propia.

El modelo de colaboración propuesto es escalonado y con riesgo mínimo en la fase inicial: (1) fase piloto de 1–2 meses con despliegue y validación extremo a extremo; (2) fase estable de 3–6 meses con apertura a grupos externos y monitorización básica; (3) fase de integración de 6+ meses con listado en el catálogo correspondiente y exploración de integración en los entornos de trabajo científico del consorcio.

7 Plan de publicación

El trabajo prevé la publicación de un manuscrito de *método* (predicción de función vía re-ranker y evaluación CAFA 6) y otro de *ingeniería* (arquitectura de la plataforma), además de la eventual coautoría en el paper retrospectivo del consorcio CAFA 6. Las revistas objetivo concretas, el orden y el calendario se discutirán en un documento aparte.

8 Cronograma orientativo

- **Corto plazo (0–3 meses):** reconstrucción limpia y reproducible del método que obtuvo la posición CAFA 6 y validación del despliegue `docker compose` en entorno externo al equipo de desarrollo.

- **Medio plazo (3–12 meses)**: primer despliegue piloto en infraestructura externa (LifeWatch, si la vía se materializa, o alternativa comparable) y *benchmarks* de reproducibilidad y *throughput* de los pilares arquitectónicos.
- **Largo plazo (12–24 meses)**: consolidación del despliegue institucional e integración del trabajo en la memoria de tesis.

El cronograma es orientativo y se ajustará en función del cierre oficial del *ranking* CAFA 6 y de la disponibilidad real de infraestructura de despliegue.

9 Colaboradores y autoría

El proyecto está co-dirigido por la Dra. Ana Rojas y el Dr. David Orellana, ambos co-directores de la tesis, quienes proporcionan supervisión científica, revisión crítica del trabajo y orientación sobre las vías de despliegue institucional. La autoría final del paper se decidirá en función de las contribuciones efectivas de cada parte al manuscrito, siguiendo las directrices ICMJE.

Se agradece explícitamente el legado intelectual de la plataforma FANTASIA, desarrollada previamente en el mismo ámbito, de la cual PROTEA hereda conceptualmente parte de su enfoque pero sobre la que se ha realizado un rediseño arquitectónico completo para resolver problemas de acoplamiento y reproducibilidad heredados.

10 Cierre

PROTEA se encuentra en un momento en el que la parte técnica y metodológica está sustancialmente madura y los resultados apuntan a una contribución significativa en el campo de la anotación funcional de proteínas. Los siguientes pasos —consolidación de los *benchmarks*, despliegue en infraestructura estable, redacción de los manuscritos y envío a las revistas objetivo— son trabajo de los próximos 12–18 meses y constituyen el núcleo de la fase final del doctorado.

Este documento se entrega como base de discusión para la revisión interna y para facilitar la presentación del proyecto a colaboradores externos —en particular, a la infraestructura LifeWatch ERIC como posible *host* del servicio en su fase de despliegue institucional.

Referencias

- [1] A. Cockburn. *Hexagonal architecture*. Artículo original disponible en alistair.cockburn.us, 2005.
- [2] R. C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] M. Ashburner et al. Gene Ontology: tool for the unification of biology. *Nature Genetics*, 25(1):25–29, 2000.
- [4] N. Zhou et al. The CAFA challenge reports improved protein function prediction and new functional annotations for hundreds of genes through experimental screens. *Genome Biology*, 20(1):244, 2019.

- [5] Z. Lin et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
- [6] M. Heinzinger et al. ProstT5: bilingual language model for protein sequence and structure. *bioRxiv*, 2023.
- [7] A. Elnaggar et al. Ankh: optimized protein language model unlocks general-purpose modelling. *bioRxiv*, 2023.
- [8] P. Jones et al. InterProScan 5: genome-scale protein function classification. *Bioinformatics*, 30(9):1236–1240, 2014.
- [9] C. P. Cantalapiedra et al. eggNOG-mapper v2: functional annotation, orthology assignments, and domain prediction at the metagenomic scale. *Molecular Biology and Evolution*, 38(12):5825–5829, 2021.
- [10] P. Törönen et al. PANNZER2: a rapid functional annotation web server. *Nucleic Acids Research*, 46(W1):W84–W88, 2018.
- [11] M. Kulmanov and R. Hoehndorf. DeepGOPlus: improved protein function prediction from sequence. *Bioinformatics*, 36(2):422–429, 2020.